

```
$ lab --phase 2 --num 1 --complete
```

Lab 1 — Phase 2

Terraform VPC + Remote State

Procédure complète — Installation, VPC, Remote State S3

Terraform

HCL

VPC

Remote State

S3 Backend

Outil

Auteur

Date

Statut

Terraform v1.15.7

Nathan

Juin 2026

Validé ✓

Table des matières

PARTIE 1 Installation et configuration

- › Installation de Terraform
- › Configuration de l'accès AWS (IAM)

PARTIE 2 Projet Terraform — VPC

- › Structure du projet
- › variables.tf
- › main.tf — Provider et VPC
- › main.tf — Subnets et routage
- › outputs.tf
- › Déploiement

PARTIE 3 Remote State S3

- › Création du bucket S3
- › Configuration backend
- › Migration du state

PARTIE 4 Validation et nettoyage

- › Checklist finale
- › Commandes de nettoyage

PARTIE 1

Installation et configuration

1. Installation de Terraform

```
bash
# Ajouter le repo HashiCorp
wget -O - https://apt.releases.hashicorp.com/gpg | \
    sudo gpg --dearmor -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg] \
    https://apt.releases.hashicorp.com $(lsb_release -cs) main" | \
    sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update && sudo apt install terraform -y

# Vérification
terraform version
# → Terraform v1.15.7 on linux_amd64
```

2. Configuration de l'accès AWS

Créer un utilisateur IAM dédié à Terraform dans la console AWS :

```
bash
Console AWS → IAM → Users → Create user
    Name      : terraform-user
    Policy    : AdministratorAccess

IAM → terraform-user → Security credentials → Create access key
    Use case : Command Line Interface (CLI)
    → Télécharger le CSV (Access Key ID + Secret Access Key)

# Sur le serveur Ubuntu :
aws configure
    AWS Access Key ID      : <Access Key ID>
    AWS Secret Access Key  : <Secret Access Key>
    Default region name    : eu-west-3
    Default output format  : json

# Vérification
aws sts get-caller-identity
# → "Arn": "arn:aws:iam::213026893036:user/terraform-user"
```

PARTIE 2

Projet Terraform — VPC

3. Structure du projet

```
• • • bash
mkdir -p ~/lab1-phase2-terraform-vpc
cd ~/lab1-phase2-terraform-vpc
touch main.tf variables.tf outputs.tf

# Créer le .gitignore
cat > .gitignore << 'EOF'
.terraform/
*.tfstate
*.tfstate.backup
.terraform.lock.hcl
EOF
```

4. variables.tf

```
variable "aws_region" {
  description = "Région AWS"
  type        = string
  default     = "eu-west-3"
}

variable "project_name" {
  description = "Nom du projet"
  type        = string
  default     = "phase2-lab1"
}

variable "vpc_cidr" {
  description = "Plage d'adresses IP du VPC"
  type        = string
  default     = "10.0.0.0/16"
}

variable "public_subnets" {
  description = "CIDRs des subnets publics"
  type        = list(string)
  default     = ["10.0.1.0/24", "10.0.2.0/24"]
}

variable "private_subnets" {
  description = "CIDRs des subnets privés"
  type        = list(string)
  default     = ["10.0.3.0/24", "10.0.4.0/24"]
}

variable "availability_zones" {
  description = "Zones de disponibilité"
  type        = list(string)
  default     = ["eu-west-3a", "eu-west-3b"]
}
```

5. main.tf — Provider et VPC

```
terraform {  
    required_providers {  
        aws = {  
            source   = "hashicorp/aws"  
            version = "~> 5.0"  
        }  
    }  
  
    backend "s3" {  
        bucket      = "terraform-state-nathan-2701"  
        key          = "phase2/lab1/terraform.tfstate"  
        region       = "eu-west-3"  
        use_lockfile = true  
        encrypt      = true  
    }  
}  
  
provider "aws" {  
    region = var.aws_region  
}  
  
# ■■■ VPC ████████████████████████████████████████████████████████████  
  
resource "aws_vpc" "main" {  
    cidr_block           = var.vpc_cidr  
    enable_dns_hostnames = true  
    enable_dns_support   = true  
  
    tags = {  
        Name     = "vpc-${var.project_name}"  
        Project = var.project_name  
    }  
}  
  
# ■■■ Internet Gateway ████████████████████████████████████████████████████████████  
  
resource "aws_internet_gateway" "main" {  
    vpc_id = aws_vpc.main.id  
  
    tags = {  
        Name     = "igw-${var.project_name}"  
        Project = var.project_name  
    }  
}
```


[illegible]

7. outputs.tf

```
output "vpc_id" {
    description = "ID du VPC créé"
    value       = aws_vpc.main.id
}

output "public_subnet_ids" {
    description = "IDs des subnets publics"
    value       = aws_subnet.public[*].id
}

output "private_subnet_ids" {
    description = "IDs des subnets privés"
    value       = aws_subnet.private[*].id
}

output "igw_id" {
    description = "ID de l'Internet Gateway"
    value       = aws_internet_gateway.main.id
}
```

8. Déploiement

```
cd ~/lab1-phase2-terraform-vpc

# Initialiser (télécharge le provider AWS)
terraform init

# Vérifier sans créer
terraform plan
# → Plan: 12 to add, 0 to change, 0 to destroy.

# Créer l'infrastructure
terraform apply
# → Taper "yes"
# → Apply complete! Resources: 12 added.

# Outputs :
# vpc_id           = "vpc-007ba14b76c14488b"
# public_subnet_ids = ["subnet-...", "subnet-..."]
# private_subnet_ids = ["subnet-...", "subnet-..."]
# igw_id           = "igw-0544fa6e0c0e72cef"
```

PARTIE 3

Remote State S3

9. Créer le bucket S3 pour le state

```
bash
Console AWS → S3 → Create bucket
  Bucket name      : terraform-state-nathan-2701
  Region           : eu-west-3
  Bucket Versioning : Enable ← historique des états
  Block Public Access : tout cocher (défaut)
→ Create bucket
```

■ Versioning — pourquoi c'est important

Le versioning S3 conserve chaque version du state. Si un apply corrompt le state, on peut restaurer une version précédente. C'est le filet de sécurité essentiel pour un remote state.

10. Configurer le backend dans main.tf

Le bloc backend est déjà dans notre main.tf (ajouté à l'étape 5). Si vous avez commencé sans le backend, voici la modification à faire dans le bloc terraform {} :

```
hcl
# Ajouter dans le bloc terraform {} :
backend "s3" {
  bucket      = "terraform-state-nathan-2701"
  key         = "phase2/lab1/terraform.tfstate"
  region      = "eu-west-3"
  use_lockfile = true
  encrypt     = true
}
```

bucket	Nom du bucket S3 créé à l'étape précédente.
key	Chemin du fichier state dans S3. Permet de gérer plusieurs projets dans le même bucket.
use_lockfile = true	Verrou S3 natif — empêche deux terraform apply simultanés.
encrypt = true	Chiffre le state en transit et au repos. Bonne pratique car le state peut contenir des infos sensibles.

11. Migrer le state local vers S3

```
bash
# Réinitialiser avec migration
terraform init -migrate-state
# → Taper "yes" pour confirmer

# Si reconfiguration nécessaire :
terraform init -reconfigure

# Vérifier que le state est dans S3
aws s3 ls s3://terraform-state-nathan-2701/phase2/lab1/
# → 2026-06-26 17:38:26    17147    terraform.tfstate
```

PARTIE 4

Validation et nettoyage

Checklist finale

✓	Terraform installé	terraform version → v1.15.7
✓	AWS configuré	aws sts get-caller-identity → terraform-user
✓	variables.tf	6 variables (région, projet, CIDRs, AZs)
✓	main.tf	Provider + backend S3 + VPC + IGW + subnets + route tables
✓	outputs.tf	vpc_id, subnet_ids, igw_id
✓	Bucket S3 créé	terraform-state-nathan-2701 avec versioning
✓	terraform init	Provider AWS + backend S3 initialisés
✓	terraform plan	12 to add, 0 to change, 0 to destroy
✓	terraform apply	12 ressources créées, outputs affichés
✓	State dans S3	aws s3 ls affiche terraform.tfstate
✓	Vérification console	vpc-phase2-lab1 visible dans AWS
✓	terraform destroy	12 ressources supprimées proprement

■ Prochaine étape — Lab 2 Terraform EC2

Lab 1 Phase 2 terminé. Tu sais créer une infrastructure VPC

complète en code Terraform avec un state partagé en équipe.

Prochaine étape : Lab 2 — ajouter EC2 et Security Group au projet Terraform.